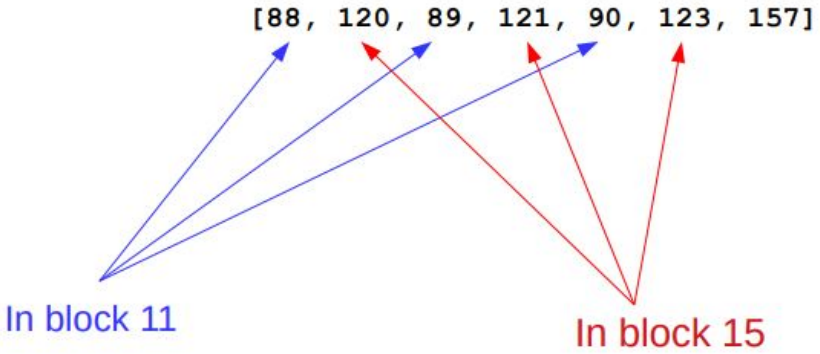
Recitation 10

Associative Caches

Direct mapped cache

In a cache with four rows and a blocksize of 8, what would you expect of the following sequence of addresses?

```
[88, 120, 89, 121, 90, 123, 157]
```



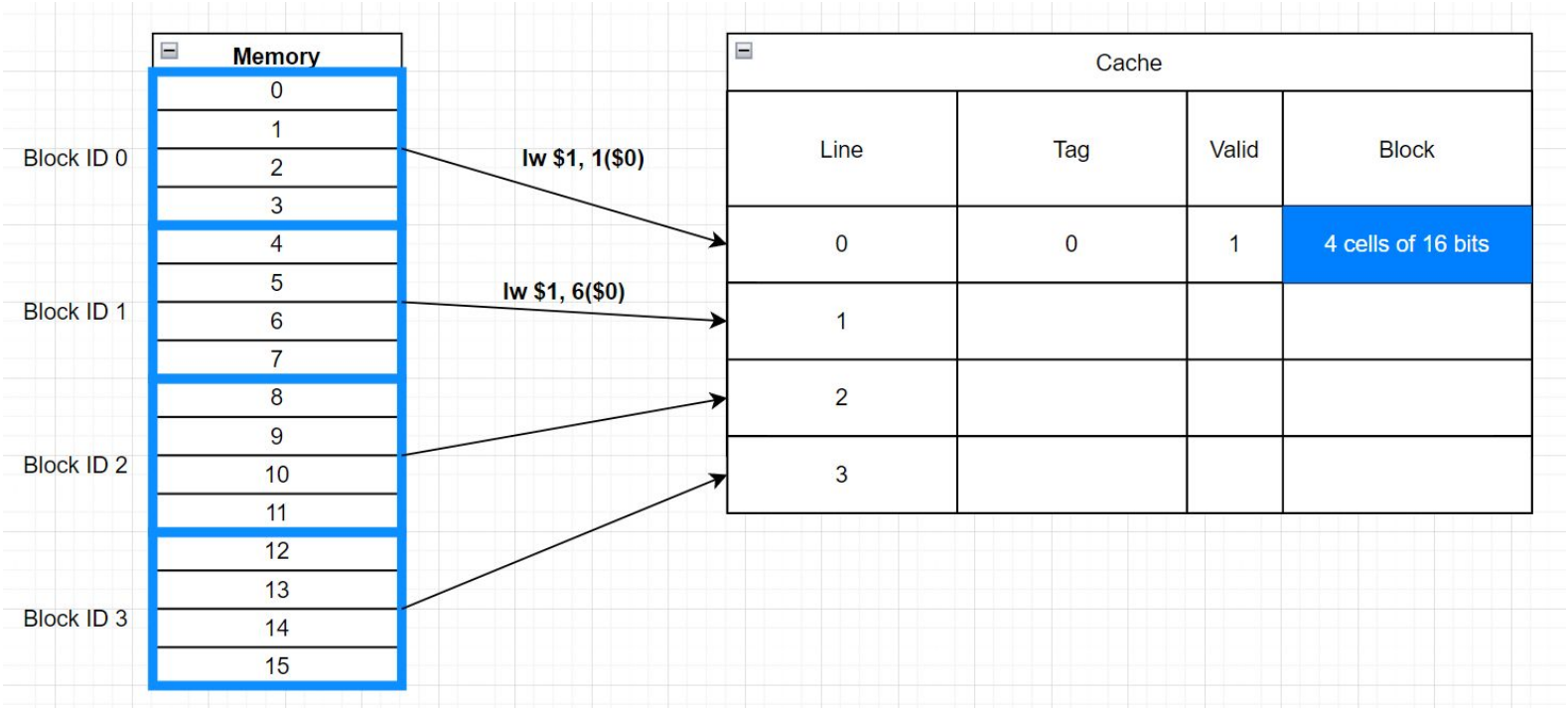
hitratio = 0

Address	Row	Tag	Hit/Miss
88	3	2	MISS
120	3	3	MISS
89	3	2	MISS
121	3	3	MISS
90	3	2	MISS
123	3	3	MISS
157	3	4	MISS

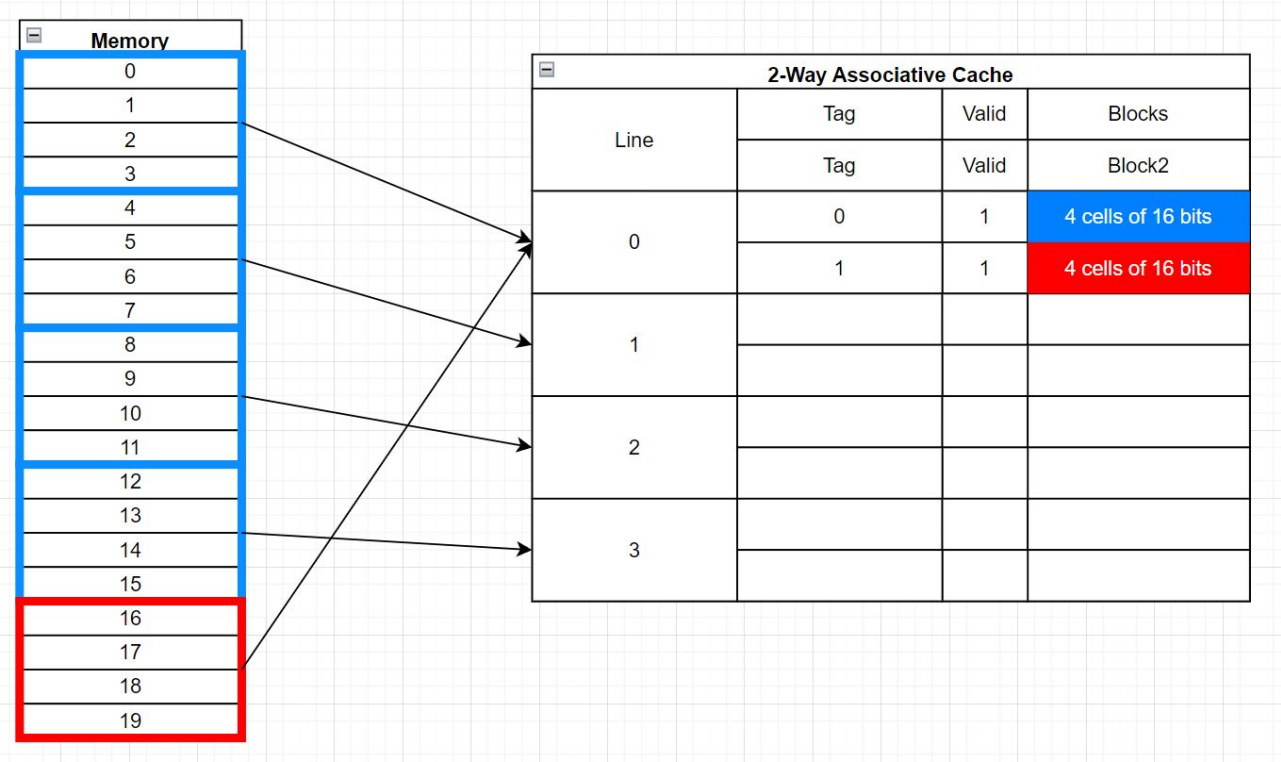
N-way Set Associative Cache

- Multiple Blocks per Row
    - Associativity(N) means # of blocks per row
    - direct mapped -> associativity = 1
- slower but fewer misses
- eviction policy (LRU)

Direct-Mapped Cache



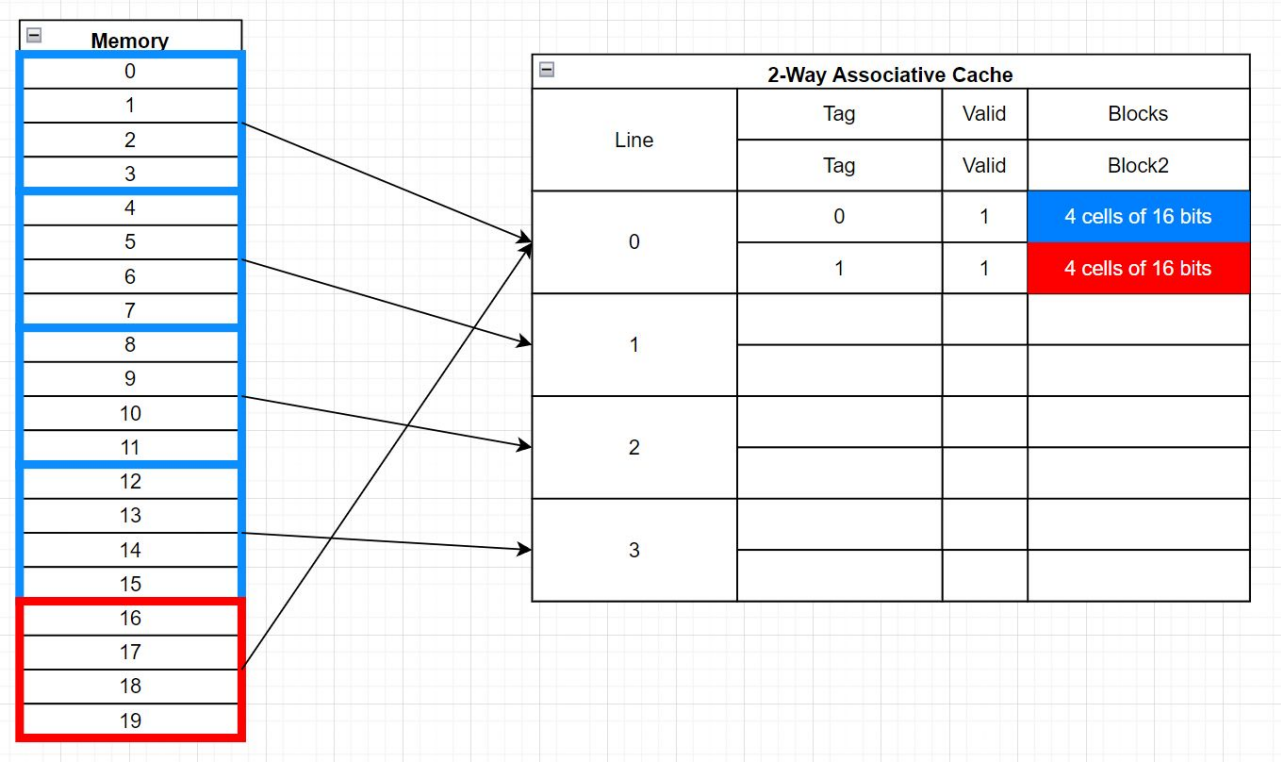
2-Way Set-Associative Cache



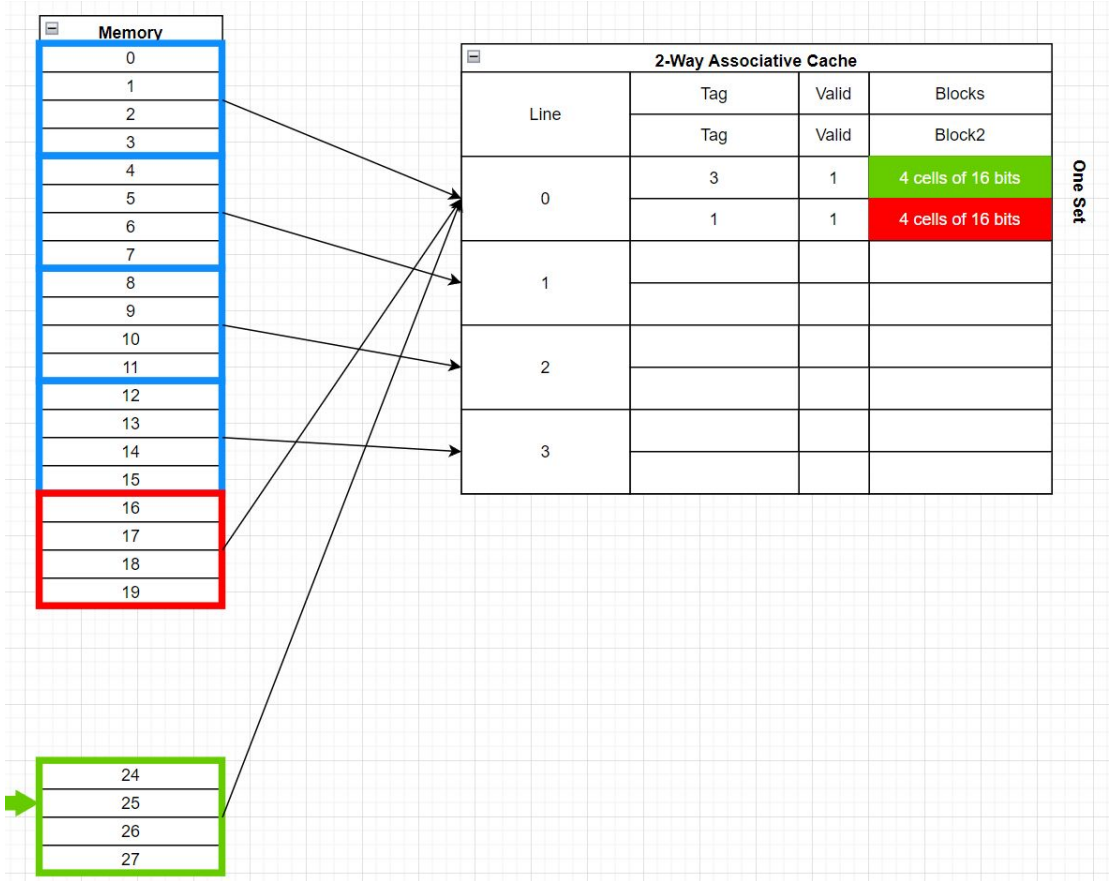
Replacement Policies

- Random replacement (RR)
  - We randomly choose one of the blocks to evict
- Least-recently used (LRU)
  - We evict the block that was accessed (read or written) least recently
  - Good strategy, but complicated to implement
- Not most-recently used (NMRU)
  - We randomly choose one of the blocks to evict, as long as it isn't the most recently used (read or written) block
- First in, first out (FIFO)
  - We evict the block that was written least recently

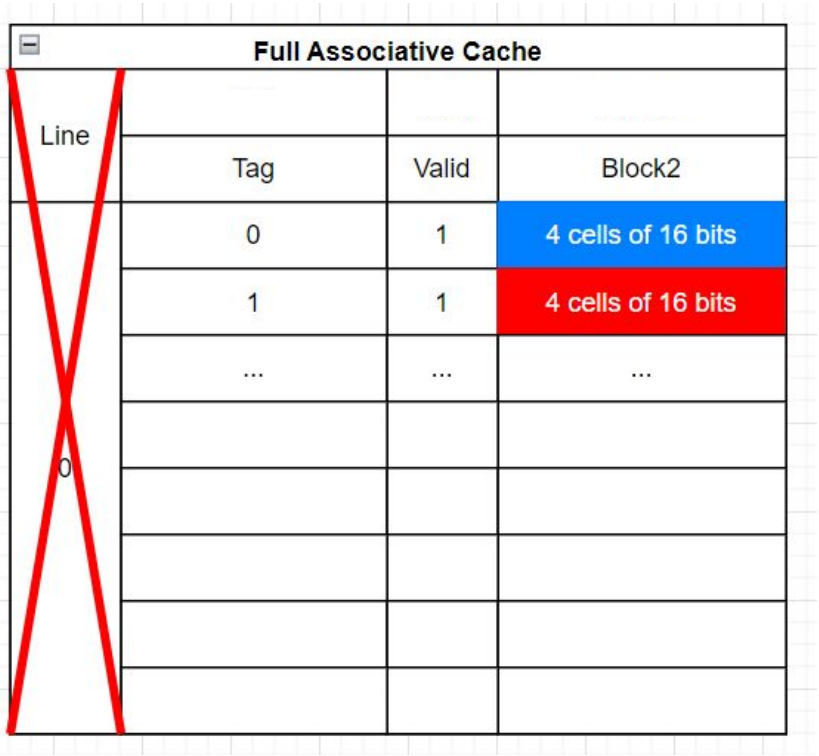
2-Way Set-Associative Cache



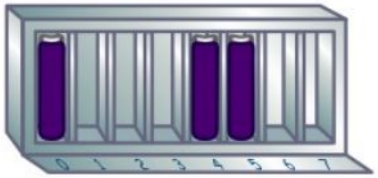
2-Way Set-Associative Cache



Fully Associative Cache



Direct Mapped



Tag	Index	Offset

A cache block can only go in one spot in the cache. It makes a cache block very easy to find, but it's not very flexible about where to put the blocks.

2-Way Set Associative

Tag	Index	Offset

This cache is made up of sets that can fit two blocks each. The index is now used to find the set, and the tag helps find the block within the set.

4-Way Set Associative

Tag	Index	Offset

Each set here fits four blocks, so there are fewer sets. As such, fewer index bits are needed.

Fully Associative

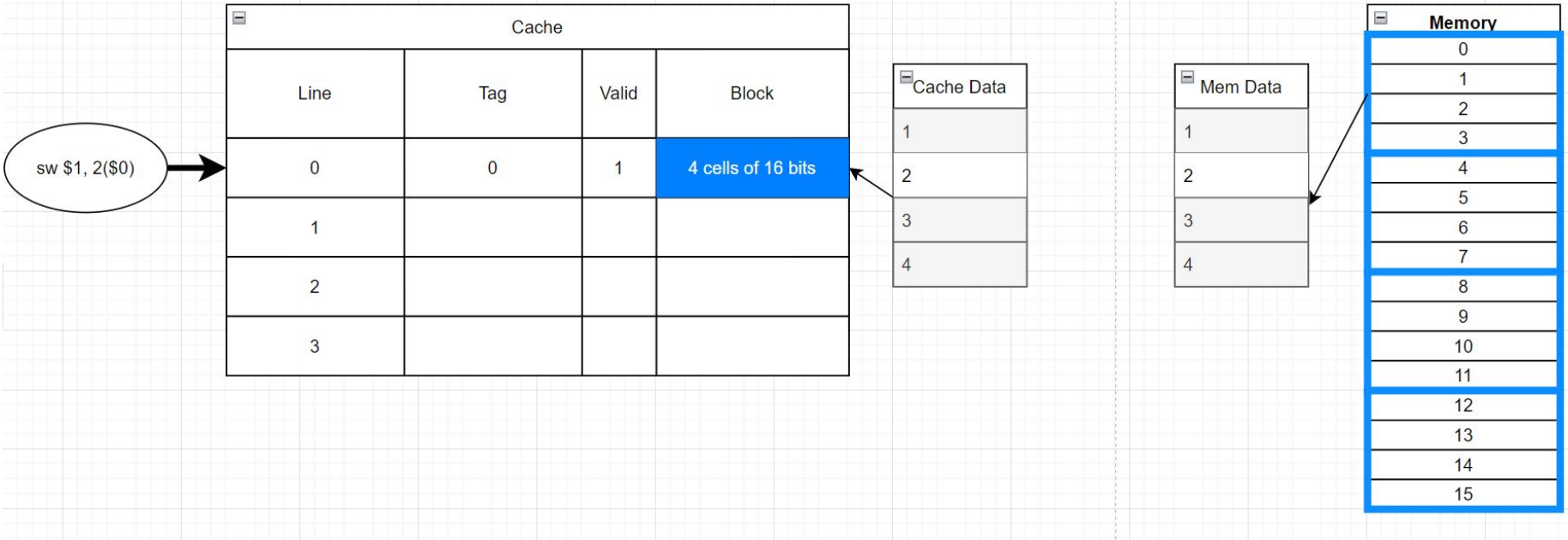
Tag	Offset

No index is needed, since a cache block can go anywhere in the cache. Every tag must be compared when finding a block in the cache, but block placement is very flexible!

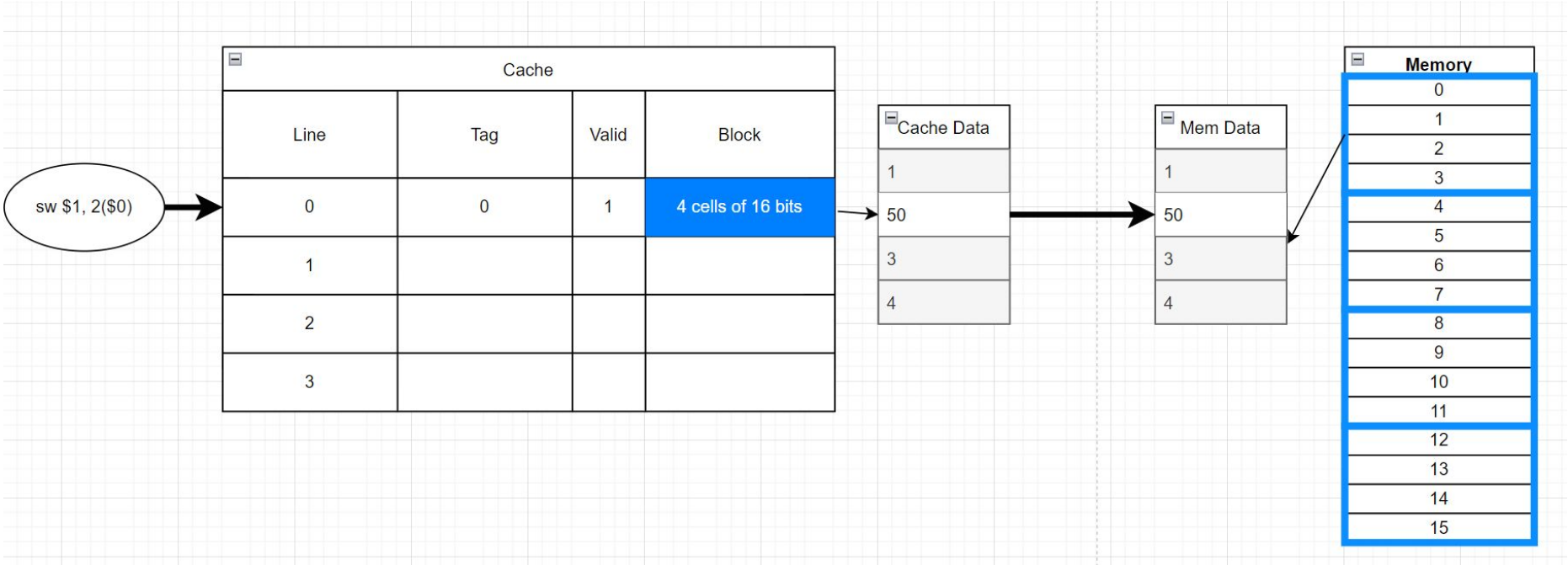
Write-Through, Write-Back

- These occur when you write to a block that is currently stored in cache
- Write-through
  - Writes will go to the cache and to RAM
  - Writes are slow, but logic is simpler
- Write-back
  - Writes will go only to the cache, and that block will be marked "dirty"
  - When a dirty block is evicted from the cache, it is written to RAM
  - Writes are faster, but implementation is complex

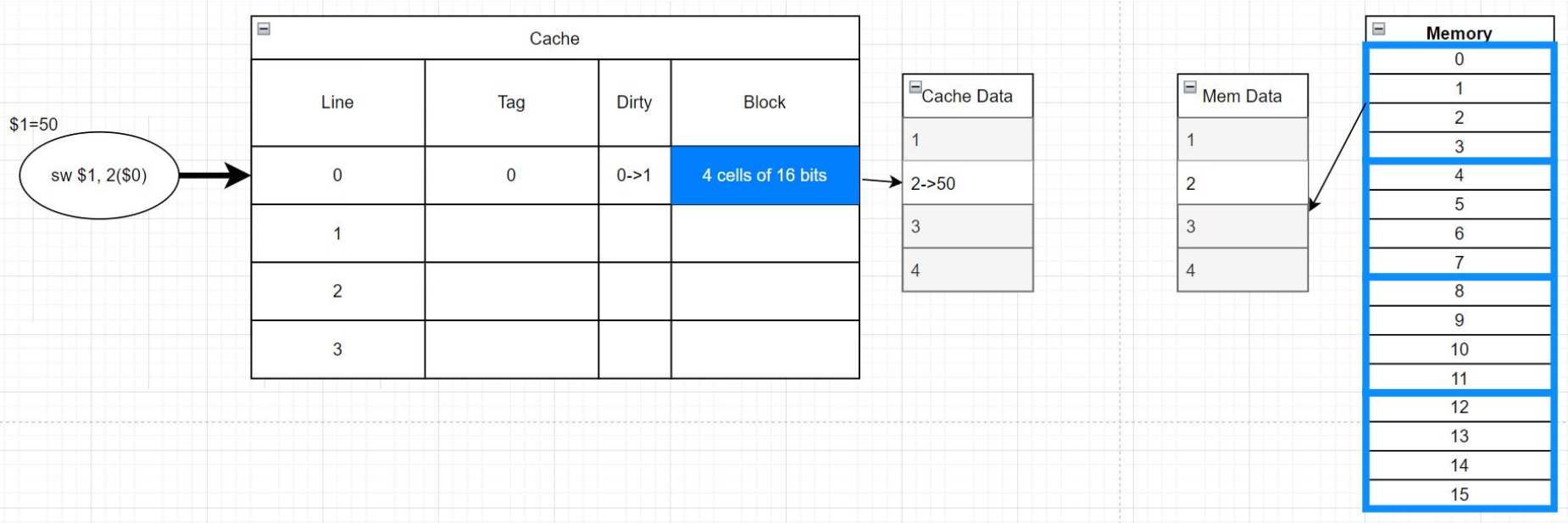
Write-Through



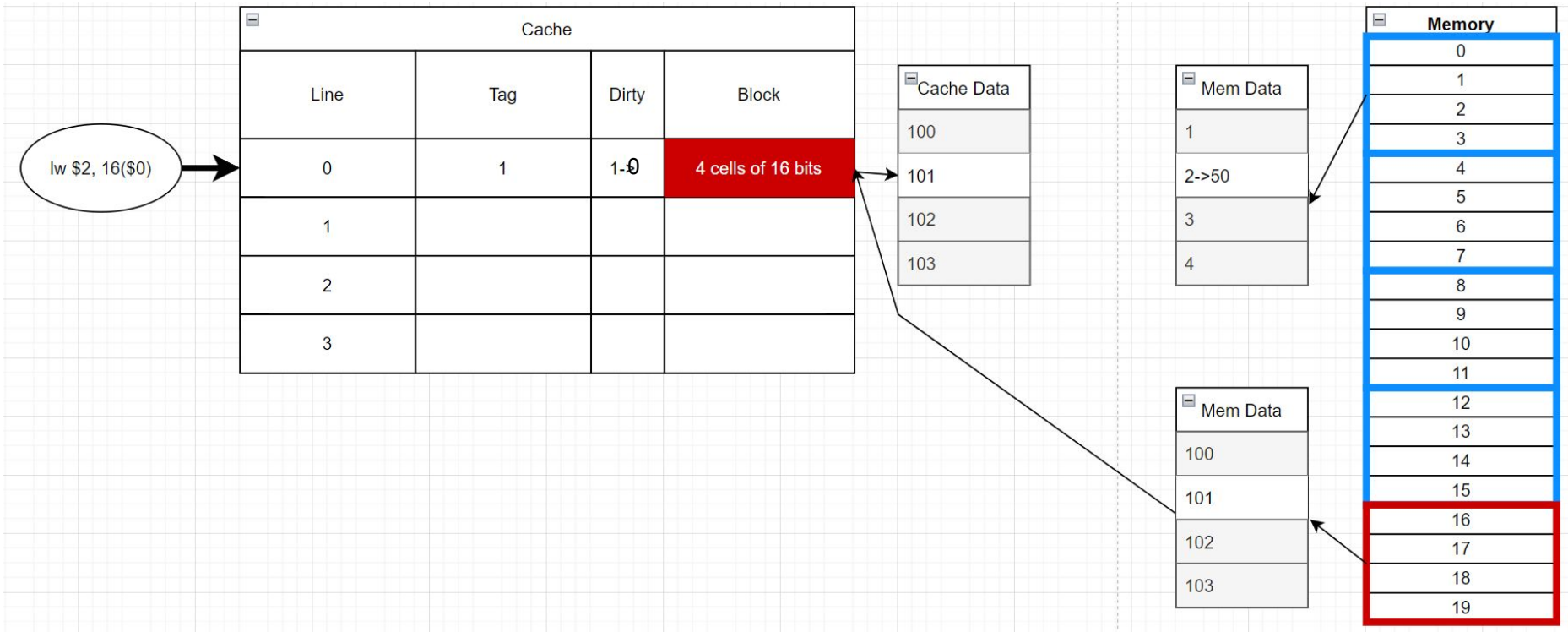
Write-Through



Write-Back


$1=50
sw $1, 2($0)
Cache
Line
Tag
Dirty
Block
0
0
0->1
4 cells of 16 bits
1
2
3
Cache Data
1
2->50
3
4
Mem Data
1
2
3
4
Memory
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

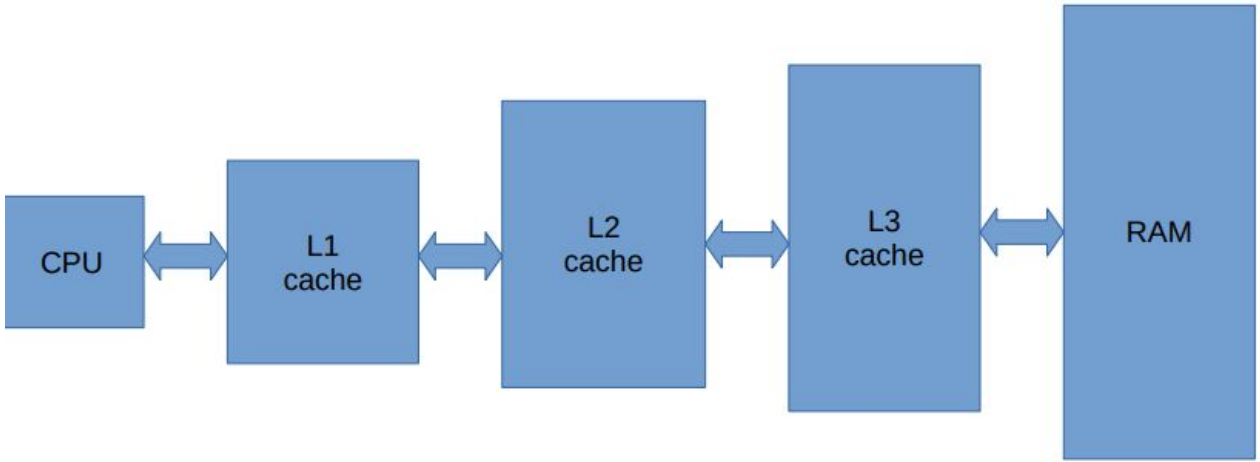
Write-Back

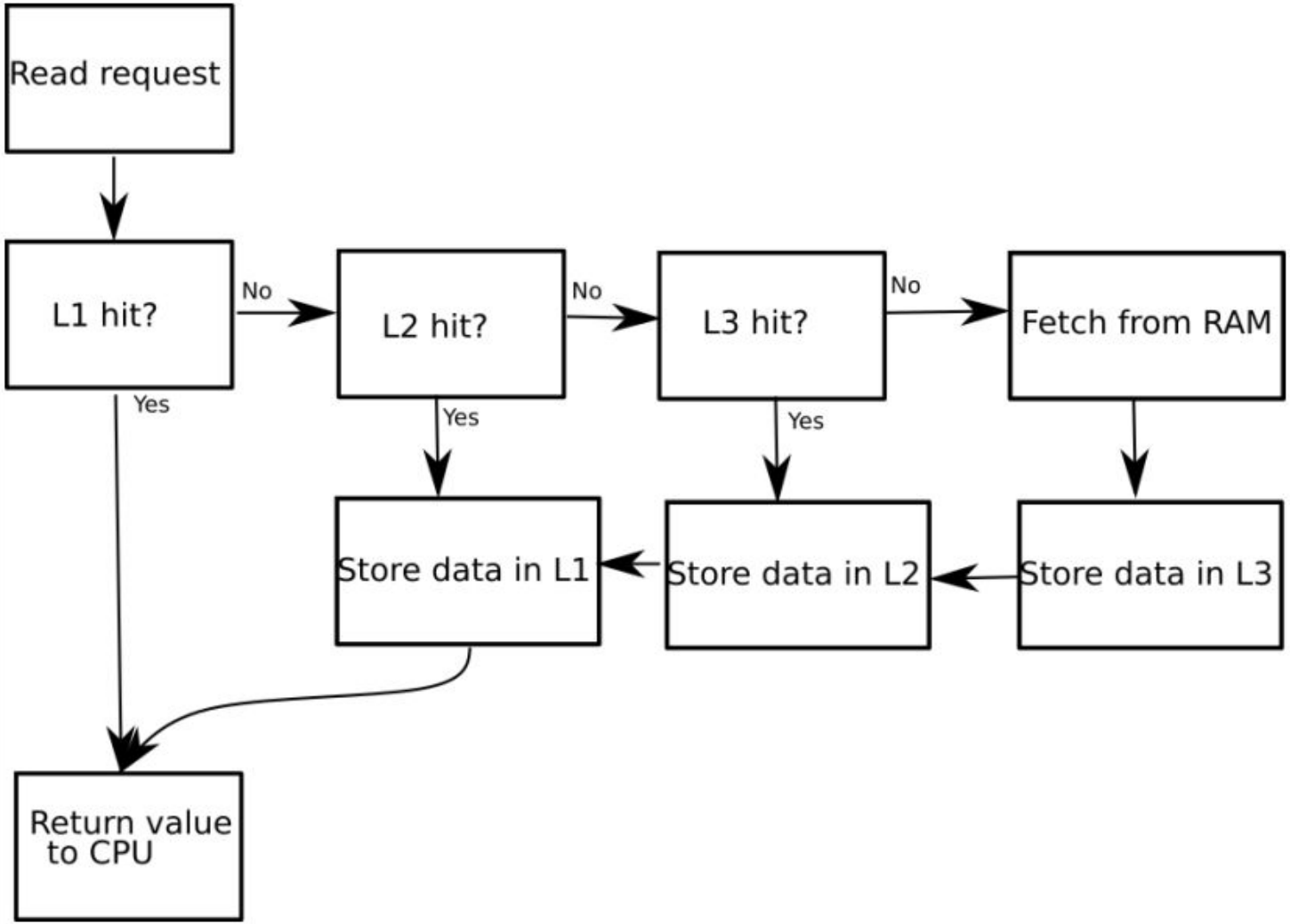

lw $2, 16($0)
Cache
Line
Tag
Dirty
Block
0
1
1->0
4 cells of 16 bits
1
2
3
Cache Data
100
101
102
103
Mem Data
1
2->50
3
4
Memory
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
Mem Data
100
101
102
103

Write-Around, Write-Allocate

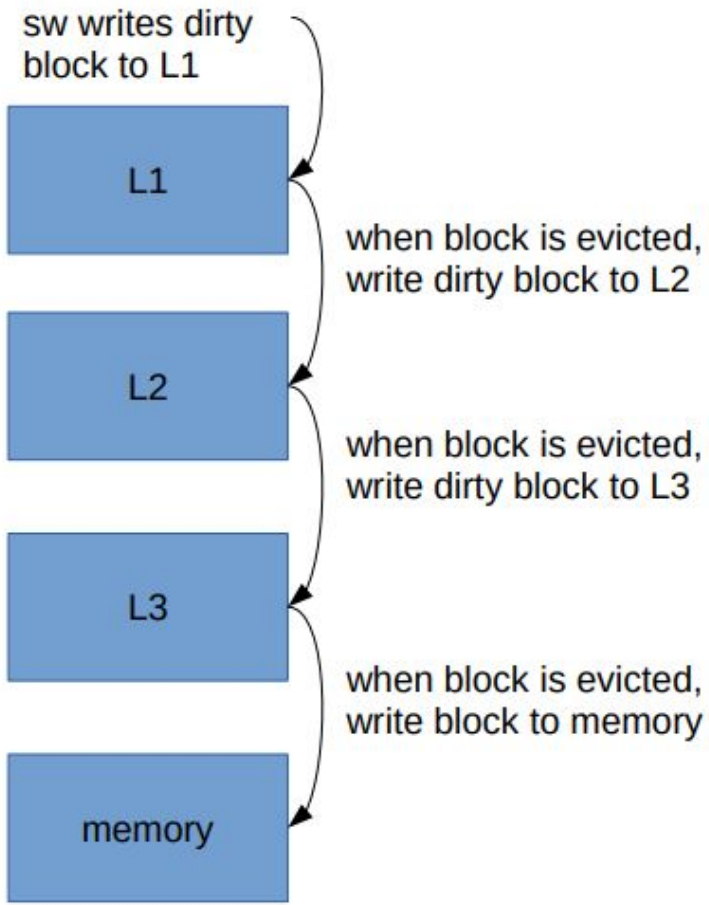
- These occur when you write to a block that is not currently stored in cache
- Write-around
  - Ignores cache and writes directly to memory
- Write-allocate
  - Write is sent to cache (new block is pulled from memory and modified)
    - Either combined with write-through or write-back policies

Multilayer Caches

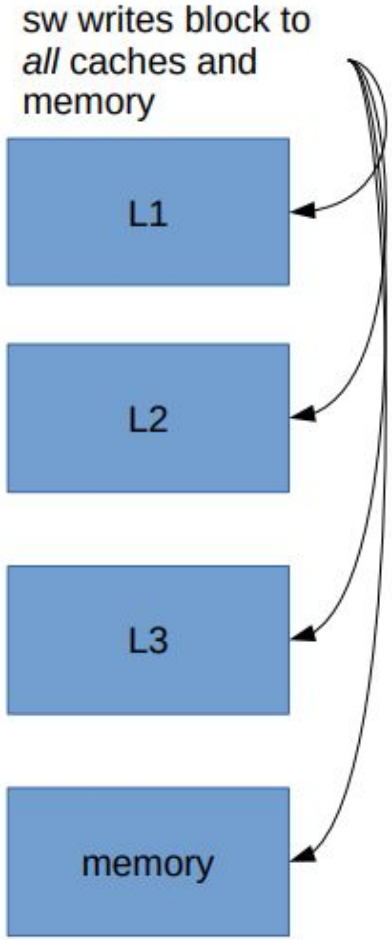



Read request
L1 hit?
No
L2 hit?
No
L3 hit?
No
Fetch from RAM
Yes
Yes
Yes
Store data in L1
Store data in L2
Store data in L3
Return value to CPU

Write-back in case of multiple caches.



Write-through in case of multiple caches.



Average Memory Access Time (AMAT) - Example

We have a processor w/ one cache. Suppose each cache access takes 8ns and each memory access takes 160ns. If we get a hit rate of 96% for a program executing a series of ‘read’ instructions, then what’s the processor’s average time per read instruction (AMAT)?

AMAT = **Hit time** + (**Miss Rate** * **Miss Penalty**)

AMAT = **8ns** + (**0.04** * **160ns**) = 14.4ns

Useful Information

- Types of caches
  - Direct mapped, n-Way Set Associative, Fully Associative
- Replacement Policies
  - Random (RR), **Least Recently Used (LRU)**, Not most-recently used (NMRU), First in, first out (FIFO)
- Write Policies
  - Block is already in cache
    - Write-Through
    - Write-Back
  - Block is not in cache
    - Write-Around
    - Write-Allocate (Must define write-through, write-back)
- Equations are located in Relevant Formulae under Recitation Slides on Brightspace

For this class, KB = KiB, MB = MiB, etc.
KiB = 2^10, MiB = 2^20, GiB = 2^30

https://www.ibm.com/docs/en/spectrum-control/5.4.2?topic=concepts-units-measurement-storage-data